

Python Modules and Libraries

Introduction

As programs grow larger, writing all code in a single file becomes difficult to manage. Python provides **Modules** and **Libraries** to organize code into reusable components.

Modules and libraries help developers:

- Reuse code
- Organize projects
- Reduce development time
- Improve maintainability
- Access powerful built-in functionalities

Almost every professional Python application uses modules and libraries.

What is a Module?

A **Module** is a Python file (.py) that contains variables, functions, classes, or executable code that can be used in another Python program.

Example

Suppose we have a file named:

calculator.py

Contents:

```
def add(a, b):  
    return a + b
```

```
def subtract(a, b):  
    return a - b
```

This file itself is called a **module**.

Why Use Modules?

Without modules:

```
# Program 1  
def add(a, b):  
    return a + b
```

```
# Program 2
def add(a, b):
    return a + b
```

The same code must be written repeatedly.

With modules:

```
import calculator

print(calculator.add(10, 5))
```

Code can be reused multiple times.

Advantages of Modules

- ✓ Code Reusability
 - ✓ Better Organization
 - ✓ Easier Maintenance
 - ✓ Reduced Redundancy
 - ✓ Faster Development
 - ✓ Improved Readability
-

Creating Your Own Module

Step 1: Create Module File

Create a file named:

mymath.py

Code:

```
def add(a, b):
    return a + b

def multiply(a, b):
    return a * b
```

Step 2: Use the Module

```
import mymath
```

```
print(mymath.add(10, 20))
print(mymath.multiply(5, 4))
```

Output:

```
30
20
```

Importing Modules

Python provides several ways to import modules.

Method 1: import module_name

```
import math
```

```
print(math.sqrt(25))
```

Output:

```
5.0
```

Method 2: import with Alias

```
import math as m
```

```
print(m.sqrt(64))
```

Output:

```
8.0
```

Method 3: Import Specific Function

```
from math import sqrt
```

```
print(sqrt(49))
```

Output:

```
7.0
```

Method 4: Import Multiple Functions

```
from math import sqrt, factorial
```

```
print(sqrt(36))  
print(factorial(5))
```

Output:

```
6.0  
120
```

Method 5: Import Everything

```
from math import *
```

Now all functions can be used directly.

```
print(sqrt(81))
```

Output:

```
9.0
```

⚠ Generally not recommended because it may create naming conflicts.

Built-in Modules in Python

Python comes with many built-in modules.

Math Module

Provides mathematical functions.

```
import math
```

sqrt()

```
import math
```

```
print(math.sqrt(16))
```

Output:

```
4.0
```

pow()

```
import math
```

```
print(math.pow(2, 3))
```

Output:

```
8.0
```

ceil()

Rounds up to nearest integer.

```
import math
```

```
print(math.ceil(4.2))
```

Output:

```
5
```

floor()

Rounds down.

```
import math
```

```
print(math.floor(4.9))
```

Output:

```
4
```

factorial()

```
import math
```

```
print(math.factorial(5))
```

Output:

```
120
```

Random Module

Used for generating random values.

```
import random
```

Random Number

```
import random
```

```
print(random.randint(1, 10))
```

Output:

7

(Random output may vary.)

Random Choice

```
import random
```

```
colors = ["Red", "Blue", "Green"]
```

```
print(random.choice(colors))
```

Output:

Blue

Datetime Module

Used for date and time operations.

```
import datetime
```

Current Date and Time

```
import datetime
```

```
now = datetime.datetime.now()
```

```
print(now)
```

Current Date

```
import datetime
```

```
today = datetime.date.today()
```

```
print(today)
```

Time Module

Provides time-related functions.

```
import time
```

```
print(time.time())
```

Delay Execution

```
import time
```

```
print("Start")
```

```
time.sleep(3)
```

```
print("End")
```

Output:

Start

(wait 3 seconds)

End

OS Module

Used to interact with the operating system.

```
import os
```

Current Directory

```
import os
```

```
print(os.getcwd())
```

Create Folder

```
import os
```

```
os.mkdir("PythonFolder")
```

Remove File

```
import os
```

```
os.remove("data.txt")
```

Statistics Module

Useful for statistical calculations.

```
import statistics
```

Mean

```
import statistics
```

```
numbers = [10, 20, 30, 40]
```

```
print(statistics.mean(numbers))
```

Output:

25

Median

```
import statistics
```

```
numbers = [10, 20, 30, 40]
```

```
print(statistics.median(numbers))
```

Output:

25

What is a Library?

A **Library** is a collection of multiple modules designed to perform related tasks.

Relationship

Library



Modules



Functions

Example:

NumPy Library



Array Module



Functions

Popular Python Libraries

Library	Purpose
NumPy	Numerical Computing
Pandas	Data Analysis
Matplotlib	Data Visualization
OpenCV	Computer Vision
TensorFlow	Artificial Intelligence
Scikit-learn	Machine Learning
Requests	API Handling
BeautifulSoup	Web Scraping

Installing Libraries

Python uses **pip** (Package Installer for Python).

Syntax

```
pip install library_name
```

Example:

```
pip install numpy
```

Install Pandas

```
pip install pandas
```

Install Matplotlib

```
pip install matplotlib
```

Using External Libraries

NumPy Example

```
import numpy as np
```

```
numbers = np.array([1, 2, 3, 4])
```

```
print(numbers)
```

Output:

```
[1 2 3 4]
```

Pandas Example

```
import pandas as pd
```

```
data = {  
    "Name": ["John", "Rahul"],  
    "Marks": [90, 85]  
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

Output:

```
   Name  Marks  
0  John    90  
1  Rahul   85
```

Matplotlib Example

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3]
y = [10, 20, 30]
```

```
plt.plot(x, y)
```

```
plt.show()
```

Checking Installed Packages

```
pip list
```

Shows all installed libraries.

Viewing Library Information

```
pip show numpy
```

Displays details about NumPy.

Updating a Library

```
pip install --upgrade numpy
```

Uninstalling a Library

```
pip uninstall numpy
```

Practical Example 1: Random Password Generator

```
import random
```

```
characters = "abcdefghijklmnopqrstuvwxyz123456789"
```

```
password = ""
```

```
for i in range(8):
    password += random.choice(characters)
```

```
print("Password:", password)
```

Practical Example 2: Digital Clock

```
import datetime

current_time = datetime.datetime.now()

print(current_time.strftime("%H:%M:%S"))
```

Practical Example 3: Calculator Using Math Module

```
import math

number = int(input("Enter Number: "))

print("Square Root:", math.sqrt(number))
print("Factorial:", math.factorial(number))
```

Mini Project: Dice Simulator

```
import random

while True:

    input("Press Enter to Roll Dice")

    print("Dice:", random.randint(1, 6))

    choice = input("Roll Again? (yes/no): ")

    if choice.lower() != "yes":
        break
```

Mini Project: Student Statistics

```
import statistics

marks = [78, 85, 92, 67, 88]

print("Average:", statistics.mean(marks))
print("Highest:", max(marks))
print("Lowest:", min(marks))
```

Best Practices

Use Meaningful Imports

✓ Good

`import math`

✗ Avoid

`from math import *`

Install Only Required Libraries

Avoid installing unnecessary packages.

Keep Libraries Updated

`pip install --upgrade package_name`

Organize Custom Modules

```
project/  
|  
├─ main.py  
├─ calculator.py  
└─ student.py
```

Real-Life Applications

Banking Systems

- Interest calculations using `math`

Data Analysis

- Pandas
- NumPy

Artificial Intelligence

- TensorFlow
- PyTorch

Web Development

- Flask

- Django

Game Development

- Pygame

Cybersecurity Tools

- Requests
 - Socket
-

Learning Outcomes

After completing this topic, students will be able to:

- ✓ Understand modules and libraries
 - ✓ Create custom modules
 - ✓ Import modules in different ways
 - ✓ Use built-in Python modules
 - ✓ Install external libraries using pip
 - ✓ Use popular libraries such as NumPy and Pandas
 - ✓ Build organized and reusable Python applications
-

Summary

Modules and libraries are fundamental building blocks of Python programming. A **module** is a single Python file containing reusable code, while a **library** is a collection of modules designed to solve specific problems.

Important concepts covered:

- Creating modules
- Importing modules
- Built-in modules (math, random, datetime, os, statistics)
- Installing libraries with pip
- Using external libraries
- Creating reusable and maintainable applications

Mastering modules and libraries allows developers to write cleaner code, reduce development time, and build powerful real-world applications efficiently.